

Neste tutorial, vamos aprender a fazer o uso da biblioteca LibMQTT na série Nexto e demonstrar o funcionamento de um projeto de exemplo.

Componentes do Tutorial

Software: MasterTool 3.71

Software: Mosquitto e CMD (Prompt de Comando)

Seções do Tutorial

1. INTRODUÇÃO
2. DEMONSTRAÇÃO DO PROJETO
 - 2.1 GVL_MQTT
 - 2.2 Espelhamentos_Entradas_MQTT
 - 2.3 Espelhamentos_Saidas_MQTT
 - 2.4 MQTT
3. CRIANDO BROKER MQTT
4. DEMONSTRAÇÃO DO FUNCIONAMENTO
 - 4.1 Alterando uma saída
 - 4.2 Recebendo o valor da entrada

1. INTRODUÇÃO



Para o desenvolvimento deste tutorial foi utilizado o CLP XP 340 da série Nexto Xpress, o software MasterTool na versão 3.71 e o CMD do Windows (Para realizar testes com o broker Mosquitto). Utilizamos como Host nosso próprio computador, mas é possível utilizar diversas plataformas destinadas à execução de aplicativos e serviços, baseada nos conceitos da computação em nuvem, como o Alibaba, Azure, etc...

O protocolo MQTT está disponível para toda Série Nexto, a partir das versões de firmware listadas abaixo.

- Série XP3xx: 1.8.11.0;
- Série NX3003 – NX3005: 1.8.11.0;
- Série NX3010 – NX3030: 1.8.3.0.

Obs1.: Para execução deste tutorial, **é necessário ter em mãos um CLP da série Nexto e que o mesmo esteja com seu firmware atualizado**. A sua versão do MasterTool da mesma forma deverá estar atualizada, sendo igual a versão 3.20 ou superior.

Obs2.: É importante ressaltar que o protocolo MQTT não funciona no modo **Simulação**. Por isso a necessidade de estar com o CLP conectado para realizar a comunicação.

Obs3.: O presente tutorial foi validado até a versão 1.6.12 do broker mosquitto, que pode ser baixado [aqui](#).

O projeto usado neste tutorial está disponível para download com o nome Project Archive MQTT. É recomendado que realize o download do mesmo e leia atentamente os comentários, pois neles estão descritos detalhadamente cada parâmetro usado no tutorial.

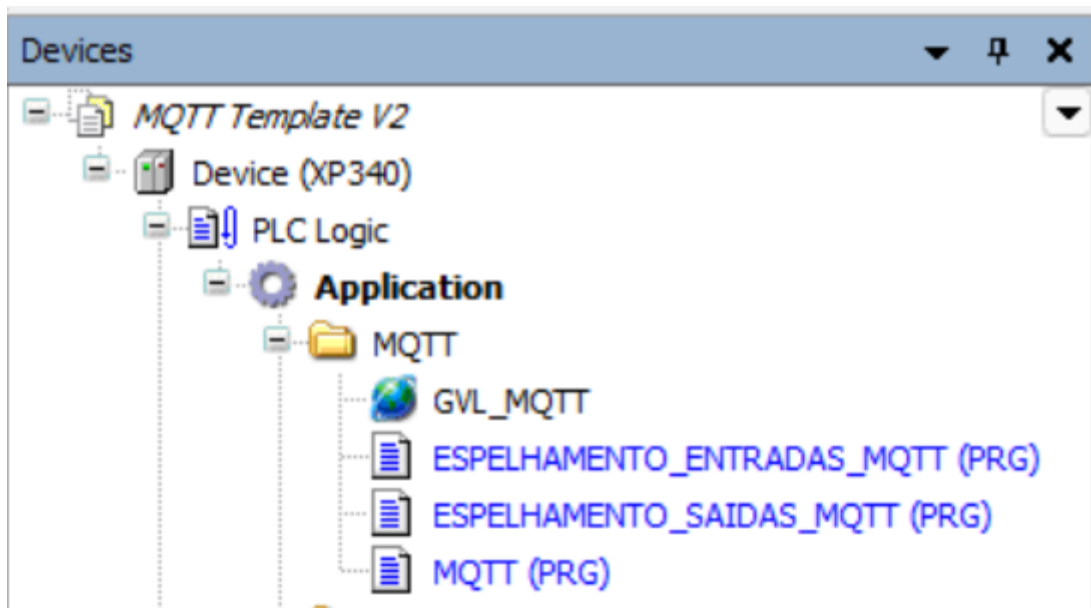
Também, disponível nos anexos deste tutorial, há um projeto com o nome Function Block MQTT. Neste projeto é realizado assinaturas e publicações de tópicos MQTT por meio de Function Block, facilitando a visualização do usuário.

Para entender mais sobre o que é MQTT, você pode conferir o artigo que escrevemos sobre o assunto e também assistir nossos Webinars:

<https://www.altus.com.br/post/301/mqtt-aplicado-a-sistemas-iot>

2. DEMONSTRAÇÃO DO PROJETO

O projeto está dividido em 3 POUs, e possui uma GVL com as declarações das variáveis utilizadas na comunicação.



Irei explicar os seus funcionamentos detalhadamente ao longo dos tópicos abaixo.

2.1 GVL_MQTT

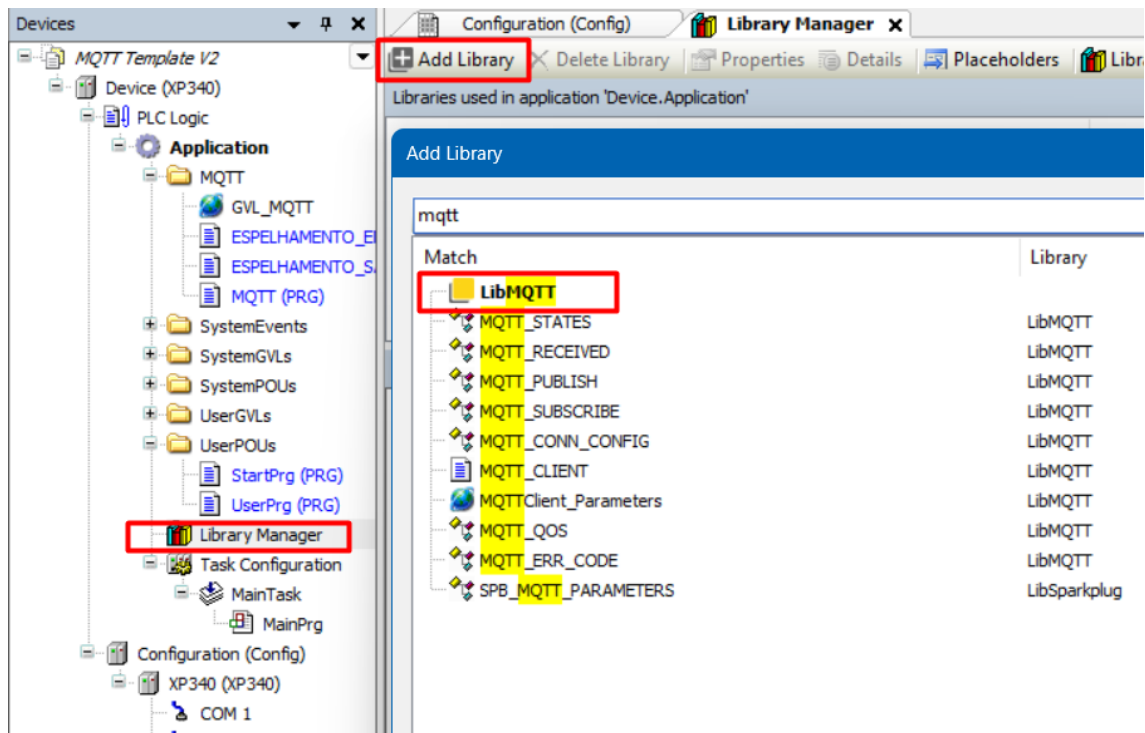
Na GVL "GVL_MQTT", encontra-se as variáveis declaradas para a comunicação MQTT. Nelas, estamos definindo os tópicos para os publishers e os subscribers, onde irão ser publicadas e lidas os valores das entradas e saídas do CLP.

```

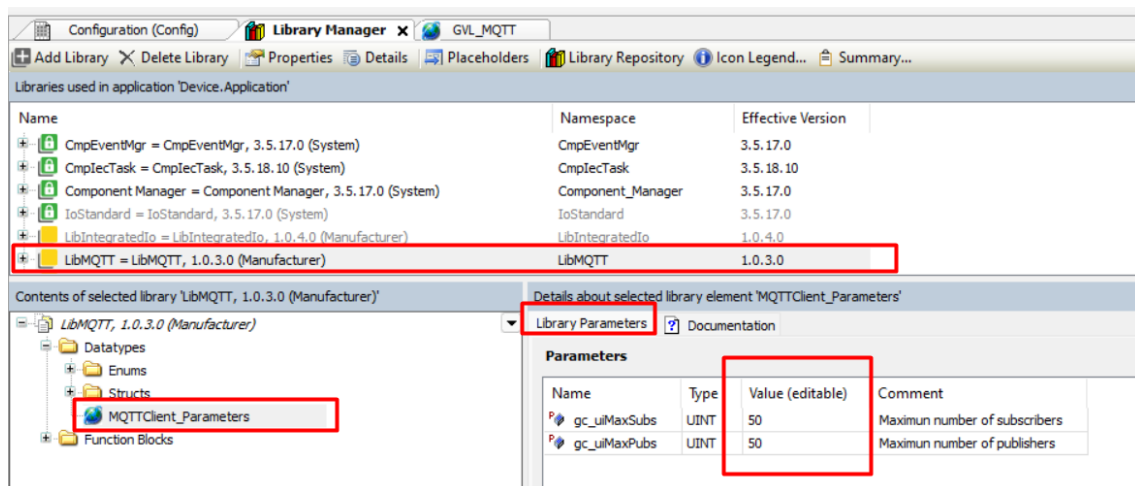
Configuration (Config) | MQTT | GVL_MQTT x
1  VAR_GLOBAL
2  Pubs : ARRAY[1..gc_uiMaxPubs] OF MQTT_PUBLISH;
3  Subs : ARRAY[1..gc_uiMaxSubs] OF MQTT_SUBSCRIBE;
4
5
6  (-----TÓPICOS DE PUBLICAÇÃO, ADICIONE, REMOVA OU MODIFIQUE CONFORME NECESSIDADE-----)
7  PUBLISH_TOPICS : ARRAY[1..gc_uiMaxPubs] OF STRING := ['XP340/I00', 'XP340/I01', 'XP340/I02', 'XP340/I03', 'XP340/I04', 'XP340/I05', 'XP340/I06', 'XP340/I07',
8  'XP340/I10', 'XP340/I11', 'XP340/I12', 'XP340/I13', 'XP340/I14', 'XP340/I15', 'XP340/I16', 'XP340/I17',
9  'XP340/AI0', 'XP340/AI1', 'XP340/AI2', 'XP340/AI3', 'XP340/AI4', 'XP340/AI5', 'XP340/AI6', 'XP340/AI7', 'XP340/AI8', 'XP340/AI9', 'XP340/AI10', 'XP340/AI11', 'XP340/AI12', 'XP340/AI13', 'XP340/AI14', 'XP340/AI15', 'XP340/AI16', 'XP340/AI17', 'XP340/AI18', 'XP340/AI19', 'XP340/AI20', 'XP340/AI21', 'XP340/AI22', 'XP340/AI23', 'XP340/AI24', 'XP340/AI25', 'XP340/AI26', 'XP340/AI27', 'XP340/AI28', 'XP340/AI29', 'XP340/AI30', 'XP340/AI31', 'XP340/AI32', 'XP340/AI33', 'XP340/AI34', 'XP340/AI35', 'XP340/AI36', 'XP340/AI37', 'XP340/AI38', 'XP340/AI39', 'XP340/AI40', 'XP340/AI41', 'XP340/AI42', 'XP340/AI43', 'XP340/AI44', 'XP340/AI45', 'XP340/AI46', 'XP340/AI47', 'XP340/AI48', 'XP340/AI49', 'XP340/AI50', 'XP340/AI51', 'XP340/AI52', 'XP340/AI53', 'XP340/AI54', 'XP340/AI55', 'XP340/AI56', 'XP340/AI57', 'XP340/AI58', 'XP340/AI59', 'XP340/AI60', 'XP340/AI61', 'XP340/AI62', 'XP340/AI63', 'XP340/AI64', 'XP340/AI65', 'XP340/AI66', 'XP340/AI67', 'XP340/AI68', 'XP340/AI69', 'XP340/AI70', 'XP340/AI71', 'XP340/AI72', 'XP340/AI73', 'XP340/AI74', 'XP340/AI75', 'XP340/AI76', 'XP340/AI77', 'XP340/AI78', 'XP340/AI79', 'XP340/AI80', 'XP340/AI81', 'XP340/AI82', 'XP340/AI83', 'XP340/AI84', 'XP340/AI85', 'XP340/AI86', 'XP340/AI87', 'XP340/AI88', 'XP340/AI89', 'XP340/AI90', 'XP340/AI91', 'XP340/AI92', 'XP340/AI93', 'XP340/AI94', 'XP340/AI95', 'XP340/AI96', 'XP340/AI97', 'XP340/AI98', 'XP340/AI99', 'XP340/AI100'];
10
11
12  (-----TÓPICOS DE ASSINATURA, ADICIONE, REMOVA OU MODIFIQUE CONFORME NECESSIDADE-----)
13  SUBSCRIBE_TOPICS : ARRAY[1..gc_uiMaxSubs] OF STRING := ['XP340/Q00', 'XP340/Q01', 'XP340/Q02', 'XP340/Q03', 'XP340/Q04', 'XP340/Q05', 'XP340/Q06', 'XP340/Q07',
14  'XP340/Q10', 'XP340/Q11', 'XP340/Q12', 'XP340/Q13', 'XP340/Q14', 'XP340/Q15', 'XP340/Q16', 'XP340/Q17',
15  'XP340/AO0', 'XP340/AO1', 'XP340/AO2', 'XP340/AO3'];
16
17
18  (-----STATUS DE PUBLICAÇÕES E MENSAGENS RECEBIDAS-----)
19  MQTT_FB_Pub_Status : ARRAY[1..gc_uiMaxPubs] OF BOOL;
20  MQTT_FB_Received_Messages : ARRAY[1..gc_uiMaxSubs] OF MQTT_RECEIVED;
21
22  (-----PAYLOADS DE PUBLICAÇÕES E ASSINATURAS-----)
23  Pubs_PAYLOADS : ARRAY[1..gc_uiMaxPubs] OF STRING;
24  Subs_PAYLOADS : ARRAY[1..gc_uiMaxSubs] OF STRING;
25
26  (-----VARIÁVEIS AUXILIARES DE PUBLICAÇÕES E ASSINATURAS-----)
27  Pubs_PAYLOADS_OLD : ARRAY[1..gc_uiMaxPubs] OF STRING;
28  END_VAR

```

OBS: Para que a comunicação MQTT funcione e você tenha acesso às variáveis e estruturas, você deve importar a biblioteca indo em Library Manager -> Add Library e procurar por MQTT

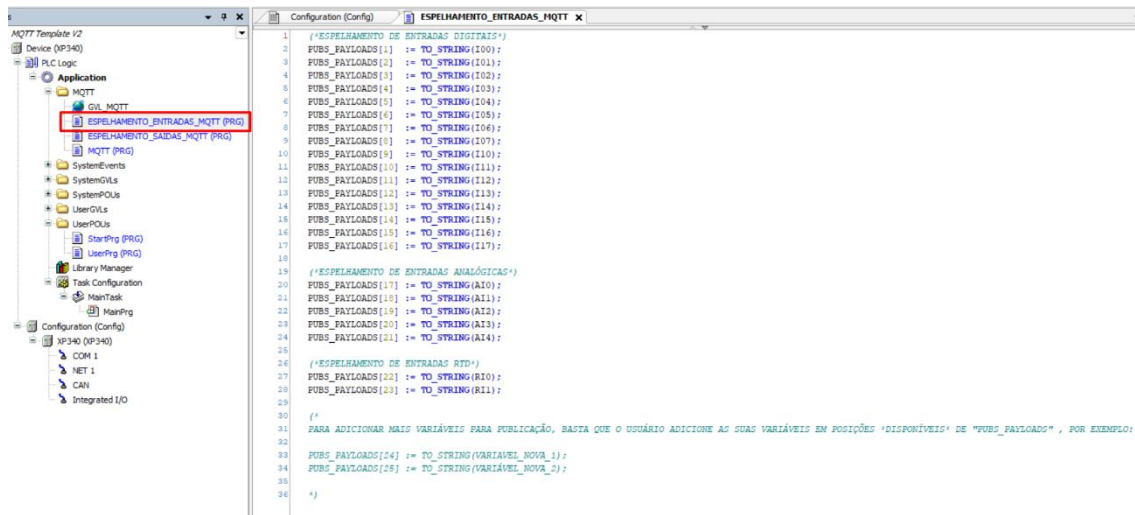


Após adicionar a biblioteca, como utilizamos diversos tópicos para a comunicação, vamos alterar o número máximo de publishers e subscribers. Para isso, clique na biblioteca adicionada, e altere os valores no caminho indicado na imagem abaixo.



2.2 Espelhamentos_Entradas_MQTT

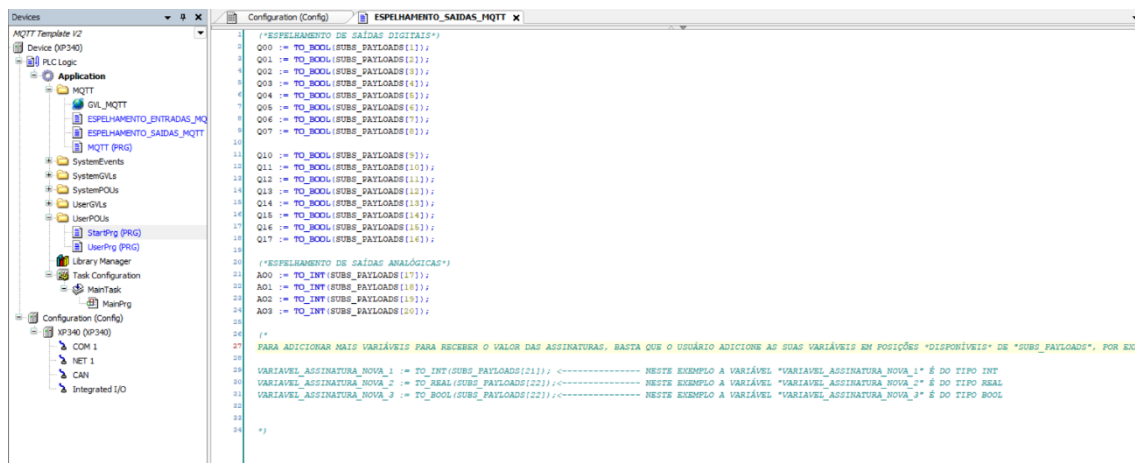
Nesta POU, estão declaradas os valores dos payloads dos publishers que serão publicados nos tópicos desejados. Para exemplificar o funcionamento, estamos criando um Payload para cada entrada do CLP, assim iremos publicar todos os seus dados, tendo cada um deles um tópico específico para o seu nome, assim como criado na GVL_MQTT.



Como escrito em forma de comentário no fim do código, você pode adicionar ali a sua lógica, adicionando mais publishers se desejado.

2.3 Espelhamentos_Saidas_MQTT

Ao contrário de como fizemos no espelhamento das entradas, no programa de espelhamento das saídas estamos declarando as variáveis de saídas digitais e analógicas para receber o seu valor por meio dos payloads dos subscribers.

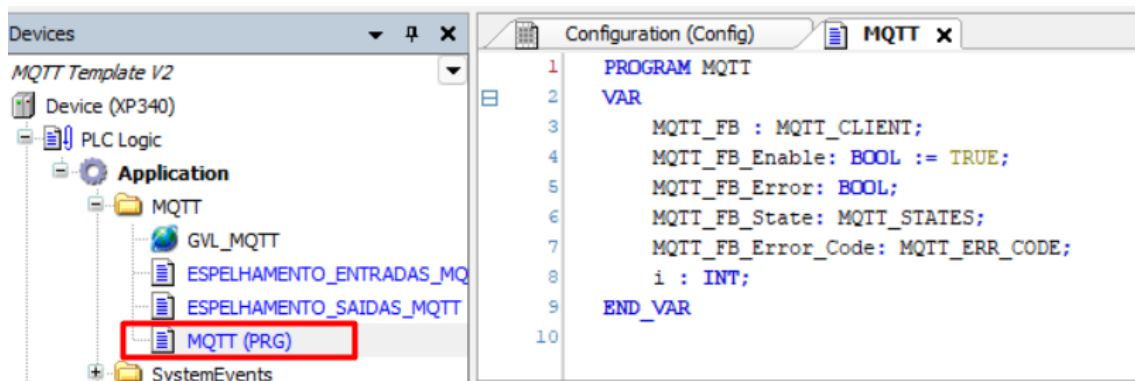


Assim como na outra POU, você pode adicionar na parte de baixo novas variáveis a serem lidas.

2.4 MQTT

Nesta POU, configuramos toda a comunicação MQTT e parametrizamos o bloco MQTT_CLIENT.

Declaramos as seguintes variáveis:



No código, vamos adicionar os seguintes códigos



Como indicado nas strings das primeiras variáveis, você deve definir o endereço de host do seu broker, que nesse caso será o IP do computador, visto que iremos hostear um broker localmente, e o ClientId, que é um nome que o CLP irá assumir na rede MQTT.

No nosso caso, considerando o IP do computador que estamos utilizando, configuramos desta forma;

```
1 (*Configurações de conexão do broker*)
2 MQTT_Fb.CONNECTION_CONFIG.sHostname := '192.168.14.114'; //Ip do seu broker
3 MQTT_Fb.CONNECTION_CONFIG.uiPort := 1883; //porta padrão do MQTT
4 MQTT_Fb.CONNECTION_CONFIG.sClientId := 'CLP_Tutorial_Altus'; //forneça um ID unico na rede MQTT
5 MQTT_Fb.CONNECTION_CONFIG.bCleanSession := TRUE; //Sugerido manter essa opção em true ca
```

Após realizar essas alterações, você pode carregar o programa no seu CLP.

3. CRIANDO BROKER MQTT

Neste tutorial, iremos hostear um broker mqtt local.

Para testar o projeto podemos utilizar o broker mosquitto, usando o CMD do Windows. O mosquitto é um broker gratuito e pode ser facilmente instalado.

Abra a pasta do mosquito localizada no diretório (C:), e digite o comando **CMD** na barra de diretórios e pressione **ENTER**.



Abra **três janelas prompt de comando**:

Na primeira execute o mosquitto.

Para executar o mosquito:

C:\Program Files\mosquitto>mosquitto -v

Na segunda você deve receber as mensagens:

C:\Program Files\mosquitto>mosquitto_sub -h localhost -p 1883 -t "Altus2/Q0"

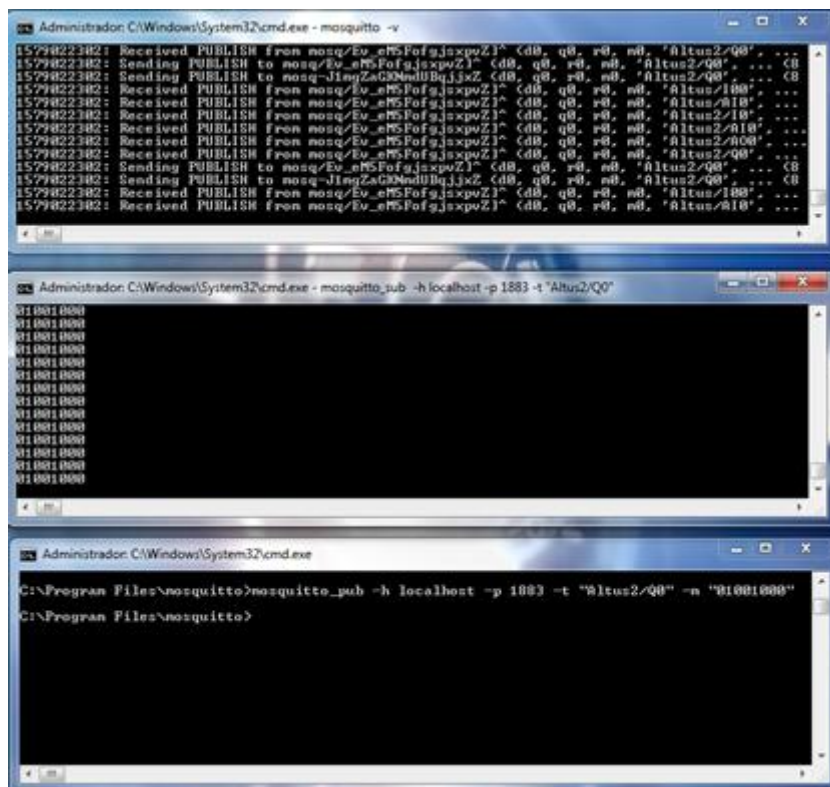
- Sub: Receber
- -h: Servidor local
- -p: porta 1883

- -t: Tópico de leitura

Na terceira você irá publicar as mensagens:

C:\Program Files\mosquitto>mosquitto_pub -h localhost -p 1883 -t "Altus2/Q0" -m "00010001"

- Pub: Publicar
- -h: Servidor local
- -p: porta 1883
- -t: Tópico de escrita
- -m: mensagem. (Deve ser digitada entre aspas duplas "00011100").



4. DEMONSTRAÇÃO DO FUNCIONAMENTO

4.1 Alterando uma saída

Para demonstrar o funcionamento alterando uma saída, enviei um comando publish ao meu broker no tópico do Q00.

```
Prompt de Comando
```

```
>mosquitto_pub -h 192.168.14.114 -t "XP340/Q00" -m "TRUE"
```

Como podemos ver no espelhamento das saídas, o CLP recebeu o comando "TRUE" no tópico "XP340/Q00" e acionou a saída Q00.

```
ESPELHAMENTO_SAIDAS_MQTT x
```

```
Device.Application.ESPELHAMENTO_SAIDAS_MQTT
```

```
1 (*ESPELHAMENTO DE SAÍDAS DIGITAIS*)
2 Q00 TRUE := TO_BOOL(SUBS_PAYLOADS[1] TRUE );
3 Q01 FALSE := TO_BOOL(SUBS_PAYLOADS[2] );
4 Q02 FALSE := TO_BOOL(SUBS_PAYLOADS[3] );
5 Q03 FALSE := TO_BOOL(SUBS_PAYLOADS[4] );
```

4.2 Recebendo o valor da entrada

Para demonstrar o funcionamento, adicionei um subscriber no tópico "XP340/AI0", e já recebi o valor atual (0).

```
Prompt de Comando - mosqu x
```

```
>mosquitto_sub -h 192.168.14.114 -t "XP340/AI0"
```

```
0
```

Após isso, alterei o valor de AI0 para receber na comunicação.

```
Devices
```

```
MQTT Template V2
```

```
Device [connected] (XP340)
```

```
PLC Logic
```

```
Application [run]
```

```
MQTT
```

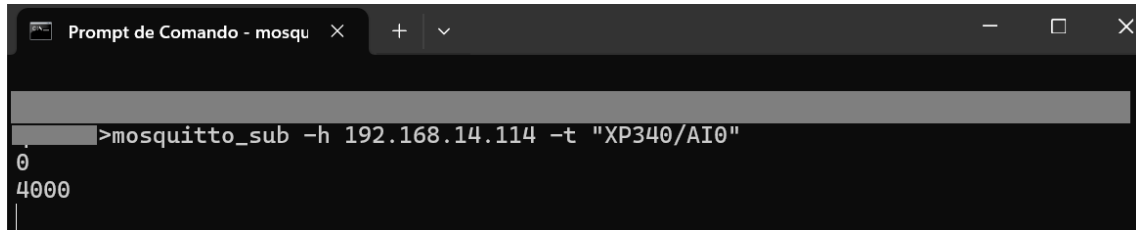
```
ESPELHAMENTO_SAIDAS_MQTT
```

```
ESPELHAMENTO_ENTRADAS_MQTT
```

```
Device.Application.ESPELHAMENTO_ENTRADAS_MQTT
```

```
8 PUBS_PAYLOADS[7] FALSE := TO_STRING(I06 FALSE );
9 PUBS_PAYLOADS[8] FALSE := TO_STRING(I07 FALSE );
10 PUBS_PAYLOADS[9] FALSE := TO_STRING(I10 FALSE );
11 PUBS_PAYLOADS[10] FALSE := TO_STRING(I11 FALSE );
12 PUBS_PAYLOADS[11] FALSE := TO_STRING(I12 FALSE );
13 PUBS_PAYLOADS[12] FALSE := TO_STRING(I13 FALSE );
14 PUBS_PAYLOADS[13] FALSE := TO_STRING(I14 FALSE );
15 PUBS_PAYLOADS[14] FALSE := TO_STRING(I15 FALSE );
16 PUBS_PAYLOADS[15] FALSE := TO_STRING(I16 FALSE );
17 PUBS_PAYLOADS[16] FALSE := TO_STRING(I17 FALSE );
18
19 (*ESPELHAMENTO DE ENTRADAS ANALÓGICAS*)
20 PUBS_PAYLOADS[17] 4000 := TO_STRING(AI0 4000 );
21 PUBS_PAYLOADS[18] 0 := TO_STRING(AI1 0 );
22 PUBS_PAYLOADS[19] 0 := TO_STRING(AI2 0 );
23 PUBS_PAYLOADS[20] 0 := TO_STRING(AI3 0 );
24 PUBS_PAYLOADS[21] 0 := TO_STRING(AI4 0 );
25
26 (*ESPELHAMENTO DE ENTRADAS RTD*)
27 PUBS_PAYLOADS[22] 0 := TO_STRING(RI0 0 );
28 PUBS_PAYLOADS[23] 0 := TO_STRING(RI1 0 );
```

E o valor atualizado foi enviado.



```
Prompt de Comando - mosqu  X + v - □ X
>mosquitto_sub -h 192.168.14.114 -t "XP340/AI0"
0
4000
```

Com isso, você está capacitado para configurar o seu CLP para comunicar com um broker MQTT e passar os dados das suas entradas e saídas para tópicos específicos, além de outros valores caso desejado